

**I made a running total
reset at fiscal year boundary
One ORDER BY swap fixed
whole sequence**

ATLIQ HARDWARE GDB0041

MYSQL | Vishnu Ram Sachin | Data Analyst



Problem Statement

DAY 12 / 15

07/08/26

- Yesterday : pivoted fy2020. fy2021 into columns with SUM(CASE())
- Leadership wants a fiscal year build up each months sales everything earlier a **running total**

The Trap ❌

- ORDER BY fiscal_month_name inside OVER() starts alphabetically , not chronologically

March is evaluated after August

Running total breaks mid fiscal year



Requirment

DAY 12 / 15

07/08/26

reset at every fiscal year



accumulated in true month order (Sep - Aug)



Partition first ,
then order correctly

The Fix

```
SUM(gsr.gross_sales) OVER (  
PARTITION BY gsr.fiscal_year  
ORDER BY gsr.date ) AS running_total
```



A Quick Analogy

DAY 12 / 15

07/08/26

Car Odometer

Every Trip

All mileage mixed together
(no partition)

Reset Every Year

Odometer rolls backs every september
(PARTITION BY fiscal_year)

Correct order

Trips counted Sep - Aug , not A - Z
(ORDER BY date)

Running Distance

each months odd onto the last
(running_total)



```
WITH gross_sales_report AS (  
  SELECT fsm.date, fsm.product_code ,  
  fsm.fiscal_year,  
  MONTHNAME(fsm.date) AS fiscal_month_name,  
  fsm.customer_code,  
  ROUND(SUM(fsm.sold_quantity * fgp.gross_price)/1000000,2) AS gross_sales  
  FROM fact_sales_monthly fsm  
  JOIN fact_gross_price fgp  
    ON fgp.product_code = fsm.product_code  
   AND fgp.fiscal_year = fsm.fiscal_year  
  GROUP BY fsm.fiscal_year , fsm.date  
  ORDER BY fsm.fiscal_year,fsm.date )  
  
SELECT  
  gsr.fiscal_year ,  
  gsr.fiscal_month_name ,  
  gsr.gross_sales,  
  SUM(gsr.gross_sales) OVER (PARTITION BY gsr.fiscal_year  
  ORDER BY gsr.date ) AS running_total  
  FROM gross_sales_report gsr  
  ORDER BY gsr.fiscal_year, gsr.date;
```



Results

DAY 12 / 15

07/08/26

fact_sales_monthly
dim_customer
fact_gross_price
dim_date
SQL File 6*

Limit to 50000 rows

```

1 • WITH gross_sales_report AS (
2     SELECT fsm.date, fsm.product_code , fsm.fiscal_year,
3     MONTHNAME(fsm.date) AS fiscal_month_name,
4     fsm.customer_code,
5     ROUND(SUM(fsm.sold_quantity * fgp.gross_price)/1000000,2) AS gross_sales
6     FROM fact_sales_monthly fsm
7     JOIN fact_gross_price fgp
8     ON fgp.product_code = fsm.product_code

```

Result Grid

Filter Rows:

Export:

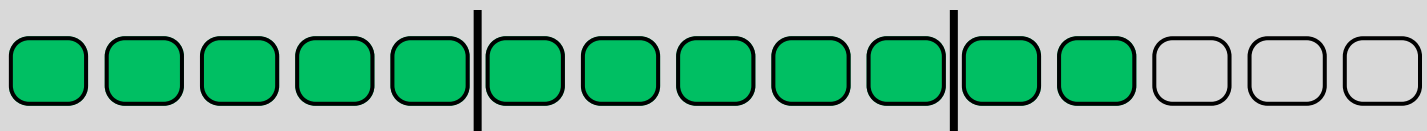
Wrap Cell Content: [A](#)

	fiscal_year	fiscal_month_name	gross_sales	running_total
▶	2018	September	4.19	4.19
	2018	October	5.30	9.49
	2018	November	7.42	16.91
	2018	December	7.54	24.45
	2018	January	4.19	28.64
	2018	February	4.04	32.68
	2018	March	4.46	37.14
	2018	April	4.25	41.39
	2018	May	4.24	45.63
	2018	June	4.13	49.76
	2018	July	4.24	54.00
	2018	August	4.32	58.32
	2019	September	15.14	15.14



DAY 12 / 15

07/08/26



I Vishnu Ram Sachin I Data Analyst

SQL THROUGH ONE DATASET CHALLENGE